

Musings on $\mathbf{uv}$ (Part I)
version: 0.1.0

Wim R.M. Cardoen, PhD

2026-05-16

Musings on uv (Part I)

1. Introduction

Why uv?

Python package management has historically required juggling several distinct tools: `pyenv` for interpreter version management, `virtualenv` or `venv` for environment isolation, `pip-tools` for dependency locking, and `pipx` for globally installed Command Line Interface (CLI) tools. Each tool solves one piece of the puzzle, but their integration is left entirely to the developer, creating configuration complexity and reproducibility challenges.

`uv` — developed by [Astral](#), the team behind the `ruff` linter — consolidates all of these responsibilities into a single, self-contained binary. It is written in [Rust](#) and is fully compatible with the existing [PyPI](#) (**P**ython **P**ackage **I**ndex) ecosystem and `pyproject.toml`-based project standards (PEP 517/518/621).

Key Features

- **Speed.** `uv` is typically 10–100× faster than `pip`, owing to its Rust implementation, parallel package downloads, and a global content-addressed cache that avoids redundant network requests. Virtual environment creation is approximately 80× faster than `python -m venv`.
- **Unified toolchain.** A single binary replaces `pip`, `pip-tools`, `virtualenv`, `pyenv`, and `pipx`.
- **Deterministic resolution.** `uv` produces a universal lockfile (`uv.lock`) that captures the complete, exact dependency graph — including transitive dependencies and platform markers — ensuring reproducible environments across operating systems and Python versions.
- **Disk efficiency.** `uv` uses a global cache combined with [hard links](#) and Copy-on-Write (CoW) semantics on supported file systems, so identical package files are never stored more than once on disk.
- **Standards compliance.** All project metadata is stored in `pyproject.toml`. No proprietary configuration formats are required.

How to load uv on CHPC Clusters

On University of Utah CHPC clusters, `uv` is available as an [LMOD](#) module. Section 2 describes the workflow in full detail.

2. In praxi

We will now discuss the use of `uv` on the CHPC clusters.

2.1. Loading the `uv` module

```
[u0253283@kp197:~]$ module load uv
[u0253283@kp197:~]$ uv -V
uv 0.11.14 (x86_64-unknown-linux-gnu)
[u0253283@kp197:~]$ which uv
/uufs/chpc.utah.edu/sys/installdir/r8/uv/0.11.14/uv
```

2.2. Initializing a new project

Rather than creating a bare virtual environment, `uv` encourages a **project-centric workflow** that improves reproducibility. A project is initialized with `uv init`, which scaffolds a standard directory structure and a `pyproject.toml` configuration file.

The following command creates a project named `myproj`, requiring Python 3.10 or later:

```
[u0253283@kp197:~]$ uv init --python 3.10 --description "My first project" -v
↪ myproj
DEBUG Searching for user configuration in:
↪ ~/uufs/chpc.utah.edu/common/home/u0253283/.config/uv/uv.toml
DEBUG uv 0.11.14 (x86_64-unknown-linux-gnu)
DEBUG Using Python version `>=3.10` from request `Python 3.10`
DEBUG Not a Git repository ~/uufs/chpc.utah.edu/common/home/u0253283/myproj
DEBUG No Python version file found in ancestors of working directory:
↪ /uufs/chpc.utah.edu/common/home/u0253283/myproj
DEBUG Writing Python versions to
↪ ~/uufs/chpc.utah.edu/common/home/u0253283/myproj/.python-version
Initialized project `myproj` at ~/uufs/chpc.utah.edu/common/home/u0253283/myproj`
```

If a project name is not provided, the project is initialized in the current working directory. The most commonly used flags are:

- `--python/-p <PYTHON>` — The minimum Python version required (sets a `>=` constraint).
- `--description <DESCRIPTION>` — A short, human-readable description of the project.
- `--verbose/-v` — Enable verbose/debug output.

After initialization, the project directory contains the following files:

```
[u0253283@kp197:~]$ ls -la myproj
total 2
drwxr-xr-x 2 u0253283 chpc 4096 May 15 14:56 .
drwxr-xr-x 2 u0253283 chpc 4096 May 15 14:56 ..
drwxr-xr-x 2 u0253283 chpc 4096 May 15 14:56 .git
-rw-r--r-- 1 u0253283 chpc 109 May 15 14:56 .gitignore
```

```
-rw-r--r-- 1 u0253283 chpc 84 May 15 14:56 main.py
-rw-r--r-- 1 u0253283 chpc 143 May 15 14:56 pyproject.toml
-rw-r--r-- 1 u0253283 chpc 5 May 15 14:56 .python-version
-rw-r--r-- 1 u0253283 chpc 0 May 15 14:56 README.md
```

Each file has a distinct role:

File	Purpose
<code>pyproject.toml</code>	The central configuration file. Stores project metadata, Python version constraints, and all declared dependencies.
<code>.python-version</code>	Records which specific Python version is pinned for this project. Consulted by <code>uv</code> (and compatible tools) to select the correct interpreter.
<code>main.py</code>	Boilerplate entry-point script.
<code>README.md</code>	Boilerplate documentation file.
<code>.git</code>	Git repository, initialized automatically.
<code>.gitignore</code>	Pre-configured to exclude the <code>.venv</code> directory and other generated files.

The initial `pyproject.toml` is minimal and human-readable:

```
[project]
name = "myproj"
version = "0.1.0"
description = "My first project"
readme = "README.md"
requires-python = ">=3.10"
dependencies = []
```

Note: In addition to standalone applications, `uv` also supports creating redistributable packages and libraries. These use cases involve additional configuration in `pyproject.toml` and will be covered in a more advanced discussion.

2.3. Managing python versions

2.3.1. Installing a python interpreter

`uv` manages its own set of Python interpreter installations, independently of the system Python. To make a specific version available to `uv`, use `uv python install`:

```
[u0253283@kp197:myproj]$ uv python install 3.12 -v
...
Python 3.12 is already installed
```

Internally, `uv` checks its local cache (located at `~/.local/share/uv/python/`) before attempting a network download. If the requested version is already cached, the command completes immediately.

Importantly, `uv python install` does **not** modify `pyproject.toml` or `.python-version`; it only ensures the interpreter is available for later use.

2.3.2. Pinning a python version to the project

To declare which Python version a project should use, run `uv python pin`:

```
[u0253283@notchpeak1:myproj]$ uv python pin 3.12 -v
...
Updated `.python-version` from `3.10` -> `3.12`
```

This command rewrites `.python-version` to record the pinned version. The version selected must satisfy the `requires-python` constraint in `pyproject.toml` (in this case, `>=3.10`, so `3.12` is valid).

Declaration vs. materialization. Pinning is a *declaration* only — it records intent in `.python-version` but does not yet create a virtual environment. The environment is materialized the first time any of the following commands runs: `uv sync`, `uv run <script>`, `uv add <package>`, or `uv venv`.

2.3.3. How `uv run` resolves and activates the environment

The `uv run` command is the primary way to execute Python within a project. Each invocation performs the following steps automatically:

1. **Locate project root.** Walk up the directory tree to find the nearest `pyproject.toml` and `.python-version` files.
2. **Select the Python interpreter.** Check, in priority order:
 - The `--python` flag (if provided on the command line).
 - The `.python-version` file.
 - The `requires-python` constraint in `pyproject.toml`.
 - `uv`'s global default interpreter.
3. **Validate or create `.venv`.** If a virtual environment does not exist, create one using the selected interpreter. If one already exists, verify it matches the required Python version.
4. **Sync dependencies.** Compare installed packages against `uv.lock`. If they are out of sync, run `uv sync` to bring the environment up to date.
5. **Execute.** Run the requested command inside `.venv`, e.g., `.venv/bin/python` for `uv run python`.

```
[u0253283@notchpeak1:myproj]$ uv run python
Using CPython 3.12.13
Creating virtual environment at: .venv
Python 3.12.13 (main, Mar 3 2026, 14:59:34) [Clang 21.1.4] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

On first invocation, `uv` creates `.venv` and also produces a `uv.lock` file alongside `pyproject.toml`. The lockfile is discussed in the next section.

2.3.4. Under the hood: the `uv.lock` file and dependency resolution

The `uv.lock` file is one of the most important outputs of the `uv` toolchain. It records a **complete, exact, reproducible snapshot** of the entire dependency graph, including all transitive (indirect) dependencies, their precise versions, cryptographic hashes, and environment markers (platform, OS, Python version). A single `uv.lock` file is *universal* — it encodes conditions for all supported platforms simultaneously, unlike a traditional `requirements.txt`, which is specific to the environment in which it was generated.

Locking and syncing are automatic: whenever `uv run`, `uv add`, or `uv sync` is invoked, `uv` checks whether the lockfile is current and updates it if necessary. The lockfile should be committed to version control so that all collaborators and CI/CD pipelines reproduce exactly the same environment.

How the dependency resolver works internally. Resolving a dependency graph is a computationally hard problem — equivalent in the worst case to Boolean satisfiability (SAT). Traditional tools like `pip` historically used a sequential backtracking algorithm, which could take many minutes on large dependency trees.

`uv` uses the **PubGrub algorithm** (originally developed for the Dart package manager and also adopted by `Cargo`, Rust’s package manager). PubGrub avoids much of the redundant backtracking of naive approaches by propagating *incompatibility constraints* eagerly: whenever a conflict is detected, it is recorded as an incompatibility and prevented from recurring, dramatically reducing the search space.

The resolution proceeds as follows:

1. **Initialize.** A virtual root package representing the project is the only “decided” package; all others are undecided.
2. **Prioritize.** The next undecided package to resolve is selected by priority: URL-pinned dependencies (git, path, file) first, then packages with `==` exact constraints, then highly conflicting packages (tracked by a conflict counter), and finally all remaining packages in breadth-first discovery order. This ordering ensures direct dependencies are resolved before transitive ones.
3. **Select a version.** Versions are tried newest-to-oldest by default (or oldest-to-newest with `resolution = "lowest"`). Versions already present in `uv.lock` or the current `.venv` are preferred to avoid unnecessary changes.
4. **Propagate.** Any new constraints implied by the selected version are added to the partial solution and checked for conflicts.
5. **Backtrack if needed.** On conflict, PubGrub derives the minimal incompatibility and backtracks to the last decision that caused it, rather than simply trying the next version sequentially.
6. **Fork for environment markers.** When a package has conflicting version requirements across different platforms or Python versions (e.g., `numpy>=2` on Python 3.11+, but `numpy>=1.16,<2` on Python 3.10), the resolver *forks* into parallel branches rather than failing. This forking strategy is what makes the lockfile universal.

The resolver itself runs on a dedicated OS thread (keeping its mutable state synchronous and simple), while network I/O for fetching package metadata runs concurrently on Tokio’s asynchronous runtime. The two sides communicate via an in-memory channel, so metadata fetching never blocks the resolution loop.

When resolution fails, `uv` can reconstruct and report the precise chain of incompatibilities that caused the conflict, producing human-readable error messages.

2.4. Adding external packages

External packages are added to the project using `uv add <package_name>`.

The command `uv add numpy scipy matplotlib` installs the `numpy`, `scipy`, and `matplotlib` packages into the current environment:

```
[u0253283@kp197:myproj]$ uv add numpy scipy matplotlib
Installed 12 packages in 7.15s
+ contourpy==1.3.3
+ cycler==0.12.1
+ fonttools==4.63.0
+ kiwisolver==1.5.0
+ matplotlib==3.10.9
+ numpy==2.4.5
+ packaging==26.2
+ pillow==12.2.0
+ pyparsing==3.3.2
+ python-dateutil==2.9.0.post0
+ scipy==1.17.1
+ six==1.17.0
```

This single command performs several operations atomically:

1. Runs the resolver to find compatible versions of `numpy`, `scipy`, and `matplotlib` and all of their transitive dependencies.
2. Updates `pyproject.toml` to record `numpy`, `scipy`, and `matplotlib` as direct dependencies:

```
dependencies = [
    "matplotlib>=3.10.9",
    "numpy>=2.2.6",
    "scipy>=1.15.3",
]
```

3. Updates `uv.lock` with the resolved versions of all packages and every package they depend on (`contourpy`, `cycler`, `fonttools`, `kiwisolver`, `packaging`, `pillow`, `pyparsing`, `python-dateutil`, `six`). `uv.lock` records not only the wheel files used in the current environment, but also the wheel files for other supported platforms.
4. Installs the packages into the project's `.venv`. The subdirectory `.venv/lib/python3.12/site-packages` contains the newly installed packages.

Because `uv.lock` captures the complete transitive graph, any collaborator who later runs `uv sync` on a fresh clone of the repository will obtain bit-for-bit identical package versions.

By default, `uv` looks for packages on [PyPI](#) (as `pip` does). This behaviour can be overridden, for

example by specifying an additional index on the command line.

Consider the `torch` package with CUDA support. The command `uv add torch` will locate `torch` on PyPI, but the PyPI distribution provides CPU support only. To obtain GPU-enabled builds, specify the PyTorch index directly:

```
[u0253283@notchpeak1:myproj]$ uv add torch --extra-index-url
↪ https://download.pytorch.org/whl/cu121
Installed 25 packages in 1m 28s
+ filelock==3.29.0
+ fsspec==2026.4.0
+ jinja2==3.1.6
+ markupsafe==3.0.3
+ mpmath==1.3.0
+ networkx==3.6.1
- numpy==2.4.5
+ numpy==2.2.6
+ nvidia-cublas-cu12==12.1.3.1
+ nvidia-cuda-cupti-cu12==12.1.105
+ nvidia-cuda-nvrtc-cu12==12.1.105
+ nvidia-cuda-runtime-cu12==12.1.105
+ nvidia-cudnn-cu12==9.1.0.70
+ nvidia-cufft-cu12==11.0.2.54
+ nvidia-curand-cu12==10.3.2.106
+ nvidia-cusolver-cu12==11.4.5.107
+ nvidia-cuspars-cu12==12.1.0.106
+ nvidia-nccl-cu12==2.21.5
+ nvidia-nvjitlink-cu12==12.9.86
+ nvidia-nvtx-cu12==12.1.105
- packaging==26.2
+ packaging==24.1
+ setuptools==70.2.0
+ sympy==1.13.1
+ torch==2.5.1+cu121
+ triton==3.1.0
+ typing-extensions==4.15.0
```

The same index configuration can be specified by editing `pyproject.toml` directly. A detailed discussion can be found [here](#).

To remove a dependency, use `uv remove <package_name>`. `uv` will uninstall the package and any of its transitive dependencies that are no longer required by any other package in the project, then update both `pyproject.toml` and `uv.lock` accordingly.

`uv` also provides a `pip`-compatible interface to ease migration from existing workflows. That said, I would recommend abandoning this option entirely and learning the native `uv` commands from the start.

2.5. How to run

There are several ways to interact with your project environment.

2.5.1. Using `uv run`

The `uv run` command was described briefly above. When invoked **within** the project directory, it automatically detects the correct Python installation. As noted earlier, the `.venv` will be created on first invocation if it does not yet exist.

```
[u0253283@notchpeak1:myproj]$ uv run python
Python 3.12.13 (main, Mar  3 2026, 14:59:34) [Clang 21.1.4 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import numpy as np
>>> np.__version__
'2.2.6'
>>> import torch
>>> torch.__version__
'2.5.1+cu121'
```

The Python executable is resolved relative to the project root. There are several ways to invoke this interpreter from outside the project directory:

- `uv run --project <path_to_project> python`

```
[u0253283@notchpeak1:~]$ uv run --project $HOME/myproj python
Python 3.12.13 (main, Mar  3 2026, 14:59:34) [Clang 21.1.4 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import sys
>>> sys.executable
'/uufs/chpc.utah.edu/common/home/u0253283/myproj/.venv/bin/python3'
```

- Reference the `.venv` directly: The interpreter binary inside `.venv/bin` can be invoked directly:

```
[u0253283@notchpeak1:~]$ $HOME/myproj/.venv/bin/python3 -c "import sys;
↪ print(sys.executable)"
/uufs/chpc.utah.edu/common/home/u0253283/myproj/.venv/bin/python3
```

2.5.2. Traditional activation

If you prefer the standard virtual environment workflow, you can activate the `.venv` that `uv` created.

- If a `.venv` does **not** exist, `uv venv` will create one.
- If a `.venv` **does** exist, `uv venv` will **overwrite** it.

To activate the project environment:

```
[u0253283@notchpeak1:~]$ source $HOME/myproj/.venv/bin/activate
(myproj) [u0253283@notchpeak1:~]$
```

Deactivation requires invoking the `deactivate` command:

```
(myproj) [u0253283@notchpeak1:~]$ deactivate  
[u0253283@notchpeak1:~]$
```

3. Documentation

uv is a relatively new project. As far as I know, no books have been published on the topic yet. The best place to learn more is the [uv documentation site](#).

4. Feedback/Comments

If you have comments or feedback, please send an email to `wcardoen \at gmail.com`.